プロジェクト詳細モーダル UI仕様書

概要

目的

プロジェクトの詳細情報（メタデータ、メンバー、ファイル一覧）を表示するモーダルコンポーネント。プロジェクト情報の確認とプロジェクトを開く操作を提供する。

使用場面

- プロジェクト一覧からプロジェクト名をクリック
- プロジェクトフォルダツリーからプロジェクトを選択
- プロジェクト検索結果から詳細を表示

アクセス権限

- **Experiment Admin（実験管理者）**: プロジェクト詳細の閲覧
- **その他のロール**: アクセス不可

画面構成

レイアウト構造

```
┌──────────────────────────────────────────────────┐
│ Project001  [進行中]                        [×]  │
│ 📁 project_folder_001                            │
├──────────────────────────────────────────────────┤
│                                                  │
│ プロジェクト情報                                 │
│ プロジェクトの説明テキスト...                    │
│                                                  │
│ 所有者              | メンバー                   │
│ 👤 user001          | user001, user002, user003  │
```

```
┌──────────────────────────────────────────────────┐
│                                                  │
│ ファイル数          │ ストレージ使用量            │
│ 📄 25 ファイル      │ 💾 15.2 MB                  │
│                                                  │
│ 作成日              │ 最終更新日                  │
│ 🗓️ 2日前            │ 🗓️ 1時間前                  │
│                                                  │
│ タグ                                             │
│ [ラマン] [XRD] [実験]                            │
│                                                  │
│ ──────────────────────────────────────────       │
│                                                  │
│ ファイル一覧 (25)                                │
│ (FileListTableコンポーネント)                    │
│                                                  │
│                          [閉じる] [プロジェクトを開く]│
└──────────────────────────────────────────────────┘
```

UI要素

1. モーダルヘッダー

- **プロジェクト名**: level 4、太字
- **ステータスタグ**: 色はステータスに応じて変化
- **フォルダ名**: `FolderOutlined` アイコン + フォルダ名（グレー）

2. プロジェクト情報セクション

- **タイトル**: 「プロジェクト情報」（level 5）
- **説明**: プロジェクトの説明テキスト（グレー）
- **メタデータグリッド**: 2列×6行のレイアウト

3. メタデータ表示項目

項目	アイコン	表示形式
所有者	`TeamOutlined`	ユーザー名
メンバー	-	カンマ区切り
ファイル数	`FileTextOutlined`	数値 + "ファイル"

ストレージ使用量	`HddOutlined`	formatFileSize()
作成日	`CalendarOutlined`	formatRelativeTime()
最終更新日	`CalendarOutlined`	formatRelativeTime()

4. タグ表示

- **ラベル**: 「タグ」（12px、グレー）
- **タグ**: Ant DesignのTagコンポーネント、wrap表示

5. ファイル一覧セクション

- **タイトル**: 「ファイル一覧 ({件数})」（level 5）
- **テーブル**: FileListTableコンポーネント

6. フッターボタン

- **閉じるボタン**: default、モーダルを閉じる
- **プロジェクトを開くボタン**: primary、プロジェクト管理画面に遷移

機能仕様

Props定義

```
interface ProjectDetailModalProps {
  open: boolean;                    // モーダル表示状態
  onClose: () => void;              // 閉じるハンドラー
  project: ProjectDetail | null;    // プロジェクト詳細
  files: FileItem[];                // ファイル一覧
}
```

データ型定義

ProjectDetail

```
interface ProjectDetail {
  id: string;                    // プロジェクトID
  name: string;                  // プロジェクト名
  folderName: string;            // フォルダ名
  description: string;           // 説明
  status: string;                // ステータス
  fileCount: number;             // ファイル数
  totalSize: number;             // 合計サイズ（バイト）
  createdAt: string;             // 作成日時
  updatedAt: string;             // 更新日時
  owner: string;                 // 所有者
  members: string[];             // メンバー一覧
  tags: string[];                // タグ一覧
}
```

FileItem

```
interface FileItem {
  id: string;                    // ファイルID
  name: string;                  // ファイル名
  path: string;                  // パス
  size: number;                  // サイズ（バイト）
  lastModified: string;          // 最終更新日時
  type: string;                  // ファイルタイプ
  isCheckedOut: boolean;         // チェックアウト状態
  checkedOutBy: string | null;   // チェックアウトユーザー
  checkedOutAt: string | null;   // チェックアウト日時
}
```

主要機能

1. ステータスタグの色取得

```
const getStatusTagColor = (status: string): string => {
  const colors: Record<string, string> = {
    planning: 'default',
    pending: 'blue',
    in_progress: 'cyan',
    completed: 'success',
  };
  return colors[status] || 'default';
};
```

ステータスと色の対応:

ステータス	色	意味
planning	default（グレー）	計画中
pending	blue（青）	保留中
in_progress	cyan（水色）	進行中
completed	success（緑）	完了

2. ステータステキストのフォーマット

```
const statusText = formatProjectStatus(project.status);
```

ユーティリティ: `formatProjectStatus()`

- ステータスコードを日本語ラベルに変換
- 例: `in_progress` → 「進行中」

3. ファイルサイズのフォーマット

```
formatFileSize(project.totalSize)
```

ユーティリティ: `formatFileSize(bytes: number)`

- バイト数を人間が読みやすい形式に変換
- 例: `15900000` → 「15.2 MB」

4. 相対時間のフォーマット

```
formatRelativeTime(project.createdAt)
formatRelativeTime(project.updatedAt)
```

ユーティリティ: `formatRelativeTime(date: string)`

- 日時を相対的な表現に変換
- 例: `2026-03-09T10:00:00Z` → 「2日前」

5. ファイルクリック処理

```
onFileClick={(file) => console.log('File clicked:', file.name)}
```

動作:

- FileListTableからファイルがクリックされた時のコールバック
- 現在は console.log のみ（将来的にファイル詳細表示やダウンロード機能を実装予定）

6. プロジェクトを開く処理

```
<Button key="open" type="primary" onClick={() => console.log('Open project')}>
  プロジェクトを開く
</Button>
```

動作:

- 現在は console.log のみ

- 将来的にプロジェクト管理画面のメインビューに遷移する機能を実装予定

モーダル設定

サイズと配置

```
<Modal
  open={open}
  onCancel={onClose}
  width={1000}
  style={{ top: 40 }}
  styles={{ body: { maxHeight: 'calc(90vh - 200px)', overflowY: 'auto' } }}
>
```

設定:

- **幅**: 1000px
- **上部位置**: 40px
- **ボディ最大高さ**: `calc(90vh - 200px)`
- **オーバーフロー**: 縦スクロール

子コンポーネント

FileListTable

用途: ファイル一覧表示

Props:

```
{
  files: FileItem[];
```

```
  onFileClick: (file: FileItem) => void;
}
```

機能:

- ファイル名、サイズ、最終更新日、チェックアウト状態を表示
- ファイルクリック時にコールバックを実行

特記事項

null チェック

```
if (!project) return null;
```

理由:

- `project` が `null` の場合、何も表示しない
- モーダルが開かれる前にデータが準備されることを前提

メンバー表示

```
<Text>{project.members.join(', ')}</Text>
```

フォーマット: カンマ + スペース区切り **例**: 「user001, user002, user003」

タグ表示

```
<Space size="small" wrap>
  {project.tags.map((tag, index) => (
    <Tag key={index}>{tag}</Tag>
```

```
  ))}
</Space>
```

動作:

- `wrap` により、横幅が不足する場合は自動的に改行
- タグが0個の場合は空のSpaceが表示される

ユーティリティ関数

formatFileSize

用途: バイト数を人間が読みやすい形式に変換

例:

- `1024` → 「1 KB」
- `1048576` → 「1 MB」
- `1073741824` → 「1 GB」

formatRelativeTime

用途: 日時を相対的な表現に変換

例:

- 1時間以内 → 「1時間前」
- 1日以内 → 「5時間前」
- 1週間以内 → 「3日前」
- それ以上 → 「2週間前」または絶対日時

formatProjectStatus

用途: ステータスコードを日本語ラベルに変換

対応表:

コード	ラベル
planning	計画中
pending	保留中
in_progress	進行中
completed	完了

getStatusColor

用途: ステータスに応じたタグの色を取得

実装: コンポーネント内で定義（`getStatusTagColor`関数）

実装ファイル情報

パス

```
frontend/src/components/project/ProjectDetailModal.tsx
```

主要な依存パッケージ

- `antd`: UI コンポーネント（Modal, Button, Typography, Tag, Divider, Space, Row, Col）
- `@ant-design/icons`: アイコン（CloseOutlined, FolderOutlined, TeamOutlined, FileTextOutlined, HddOutlined, CalendarOutlined）
- `react`: React本体

ユーティリティ

- `formatters`: formatFileSize, formatRelativeTime, formatProjectStatus, getStatusColor

子コンポーネント

- `FileListTable`：ファイル一覧表示

変更履歴

バージョン	日付	変更内容	担当者
1.0	2026-03-11	実装ベースで仕様書を新規作成（実装ファイル: ProjectDetailModal.tsx）	Cline

---

関連ドキュメント

- プロジェクト管理画面
- プロジェクトフォルダツリー
- ファイル一覧テーブル
- プロジェクト作成モーダル